SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE:CSA1104

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL2, BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 6 **Total Marks:30**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I _Dharshini/192521408_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Dharshini

Annexure A

Short Answer Test

1.Suppose a software system is developed for a **Smart Hotel Management System** where guests can book rooms, cancel reservations, and make online payments.

(a) Explain the following object-oriented concepts:

- Class
- Object
- Inheritance
- Encapsulation
- Dynamic Binding

(b) Describe the following object relationships:

- Association
- Dependency
- Composition

(c) Explain how these concepts improve software maintainability and code reusability in a hotel management system.
(5 Marks)

2.Consider developing a **Smart Education Management System**.

(a) Explain the following object-oriented design principles:

- Abstraction
- Encapsulation
- Information Hiding

(b) Describe the following SOLID principles:

- Single Responsibility Principle (SRP)
- Dependency Inversion Principle (DIP)

(c) Explain how these principles improve software scalability and maintainability.
(5 Marks)

3.Suppose a software company is developing a **Smart Pharmacy Management System**.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- Feasibility Study
- Requirement Analysis
- System Design
- Implementation
- Testing
- Maintenance

(b) Describe the significance of each SDLC phase.
(c) Explain how SDLC helps reduce project risks and improve software quality.
(5 Marks)

4.Consider a project for developing a **Smart E-Commerce System**, where customer requirements frequently change due to market demands.

(a) Explain the following SDLC models:

- Waterfall Model
- V-Model
- Incremental Model
- Agile Model

(b) Compare the Waterfall Model and Incremental Model.
(c) Recommend the most suitable SDLC model for this project with justification.
(5 Marks)

5.Suppose a software analyst is designing a **Smart Hospital Information System**.

(a) Explain the role of UML in object-oriented software design.
(b) Describe the following UML diagrams:

- Use Case Diagram
- Sequence Diagram
- Communication Diagram
- Activity Diagram

(c) Explain how UML diagrams support software analysis, design, and documentation.
(5 Marks)

6.Consider developing a **Smart Railway Reservation System**.

(a) Explain the concept of object-oriented software design.

(b) Compare the structured software development approach with the object-oriented software development approach.
(c) Explain how object-oriented design improves software flexibility, scalability, and reusability.
(5 Marks)

Answer:

1. Smart Hotel Management System

(a) Object-oriented concepts

- Class: A blueprint used to create objects. For example, `Room` defines room number, type, price, and availability.
- Object: An instance of a class. For example, Room 101 is an object of the `Room` class.
- Inheritance: Allows one class to acquire properties and methods from another class. For example, `PremiumRoom` inherits from `Room`.
- Encapsulation: Combines data and methods into a single unit and restricts direct access to data. For example, a guest's payment details can be protected using private variables.
- Dynamic Binding: The method to be executed is determined at runtime. For example, different room types may implement their own `calculatePrice()` method.

(b) Object relationships

- Association: Represents a relationship between objects. For example, a Guest makes a Reservation.
- Dependency: Occurs when one class temporarily uses another class. For example, a Reservation class uses a PaymentService to process payment.
- Composition: Represents a strong whole-part relationship. For example, a HotelRoom object may contain RoomAmenity objects as its components in a simplified design.

(c) Maintainability and code reusability

Object-oriented concepts improve maintainability by organizing the system into separate classes. Encapsulation protects data, while inheritance allows existing code to be reused. Dynamic binding supports different room pricing methods without changing the main booking logic. These features make the hotel system easier to update, test, and expand.

2. Smart Education Management System

(a) Object-oriented design principles

- Abstraction: Displays essential features while hiding unnecessary implementation details. For example, a student can view marks without knowing how they are calculated internally.
- Encapsulation: Combines data and methods in one class and controls access to them. For example, a Student class manages student details securely.

- Information Hiding: Prevents other classes from directly accessing internal implementation details. For example, examination marks can be private and modified only through authorized methods.

(b) SOLID principles

- Single Responsibility Principle (SRP): A class should have only one main responsibility. For example, a `StudentRecord` class manages student records, while a `NotificationService` sends notifications.
- Dependency Inversion Principle (DIP): High-level modules should depend on abstractions rather than concrete implementations. For example, a result management module can use a notification interface instead of depending directly on a particular email service.

(c) Scalability and maintainability

These principles make the system easier to modify, test, and extend. Abstraction simplifies interaction with system features, while encapsulation and information hiding protect internal data. SRP separates responsibilities, and DIP allows services to be replaced without major changes. Therefore, new modules such as online examinations and attendance tracking can be added more easily.

3. Smart Pharmacy Management System

(a) Phases of the Software Development Life Cycle (SDLC)

1. Feasibility Study: Determines whether the project is technically, economically, and operationally practical.
2. Requirement Analysis: Collects and documents requirements such as medicine stock management, billing, prescriptions, and expiry-date tracking.
3. System Design: Defines the software architecture, database, user interfaces, and modules.
4. Implementation: Converts the design into working code using a suitable programming language.
5. Testing: Identifies defects and verifies that the software meets the specified requirements.
6. Maintenance: Fixes errors, improves performance, and updates the software after deployment.

(b) Significance of each phase

Each phase contributes to successful development. Feasibility prevents investment in impractical projects. Requirement analysis ensures user needs are understood. Design provides a clear structure, implementation creates the software, testing checks its correctness, and maintenance keeps the system reliable over time.

(c) Risk reduction and software quality

SDLC follows a systematic development process that identifies problems early. Clear requirements reduce misunderstandings, design helps prevent architectural issues, and testing

detects defects before deployment. Maintenance resolves problems found during actual use. Thus, SDLC reduces project risks, controls costs, and improves software quality.

4. Smart E-Commerce System

(a) Explain the SDLC models

- Waterfall Model: A sequential model in which each phase must be completed before the next phase begins. It is suitable when requirements are clear and stable.
- V-Model: An extension of the Waterfall Model in which each development phase has a corresponding testing phase. It emphasizes verification and validation.
- Incremental Model: The system is developed and delivered in smaller functional parts. Each increment adds new features to the existing system.
- Agile Model: An iterative approach in which software is developed in short cycles, with continuous customer feedback and frequent releases.

(b) Compare Waterfall Model and Incremental Model

Waterfall Model	Incremental Model
Follows a sequential process.	Develops the system in smaller increments.
Requirements are generally fixed at the beginning.	Requirements can be refined between increments.
Working software is usually delivered near the end.	Working software is delivered in stages.
Changes can be difficult and costly.	Changes can be incorporated more easily between increments.

(c) Recommend a suitable SDLC model

The Agile Model is the most suitable for the Smart E-Commerce System because customer requirements frequently change according to market demands. Agile supports continuous feedback, frequent releases, and quick adaptation to changing requirements. New features such as digital payment options, product recommendations, and delivery tracking can be developed and improved in successive iterations.

5. Smart Hospital Information System

(a) Role of UML in object-oriented software design

Unified Modeling Language (UML) is a standard visual modeling language used to represent the structure and behavior of a software system. It helps developers understand requirements, identify classes and their relationships, communicate design ideas, and document the system before implementation.

(b) UML diagrams

- Use Case Diagram: Shows the interactions between users and the system. For example, doctors view patient records, and patients book appointments.
- Sequence Diagram: Shows the sequence of messages exchanged between objects over time. For example, a patient requests an appointment, the system checks availability, and the booking is confirmed.
- Communication Diagram: Shows how objects interact through messages and emphasizes their relationships. For example, Patient, Appointment, and Doctor objects communicate during appointment booking.
- Activity Diagram: Represents the workflow of activities and decisions. For example, patient registration proceeds through entering details, validating information, and saving the record.

(c) How UML supports software development

UML diagrams help analysts understand requirements and identify missing functionality. They provide a clear design for developers before coding begins. They also improve communication among team members and serve as documentation for future maintenance. Therefore, UML reduces misunderstandings and supports the development of a reliable hospital information system.

6. Smart Railway Reservation System

(a) Concept of object-oriented software design

Object-oriented software design (OOSD) is an approach that organizes a software system around objects containing data and methods. It uses concepts such as classes, objects, inheritance, encapsulation, abstraction, and polymorphism.

In a Smart Railway Reservation System, classes may include `Passenger`, `Train`, `Ticket`, `Reservation`, and `Payment`. These classes work together to provide functions such as train searching, ticket booking, cancellation, and payment processing.

(b) Structured approach vs. object-oriented approach

Structured Approach	Object-Oriented Approach
Focuses mainly on functions and procedures.	Focuses on objects containing data and methods.
Data and functions are generally handled separately.	Data and methods are grouped into classes.
Reusability is mainly achieved through functions.	Reusability is supported by inheritance and composition.
Changes to shared data may affect several functions.	Encapsulation helps control access to data.
Suitable for procedure-oriented applications.	Suitable for modular and extensible applications.

(c) Flexibility, scalability, and reusability

Object-oriented design improves flexibility by allowing individual classes to be modified without changing the entire system. It improves scalability by allowing new features, such as seat selection, live train status, and online cancellation, to be added through separate modules. It improves reusability through inheritance, composition, and reusable classes.

For example, different ticket types can share common properties through a common class while providing their own pricing rules. Thus, object-oriented design makes the railway reservation system easier to maintain, extend, and reuse.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps